

## How to Make AI Agents

### 1. What an AI agent is

An AI agent is a system that can **reason, choose actions, use tools, and iterate** toward a goal instead of only generating a single response. In practice, an agent usually combines a model, tools, a system prompt, memory, and a loop that keeps checking whether more action is needed.

At an advanced level, the difference between a chatbot and an agent is that a chatbot replies, while an agent can decide to do something, call a function, inspect results, and continue working.

### 2. Core building blocks

Most AI agents have these parts:

- Model: the LLM that thinks.
- System prompt: the role and behavior rules.
- Tools: functions the agent can call.
- Memory: short-term or long-term context.
- Agent loop: repeated think-act-observe cycle.
- Guardrails: safety and limits.

If you understand these six pieces, you can build most beginner-to-advanced agents in Python.

### 3. Agent types

There are several common forms of agents:

- Rule-guided agents, which follow fixed logic.
- Tool-using agents, which call APIs or functions.
- Workflow agents, which follow a structured sequence.
- Multi-agent systems, where specialized agents work together.
- Autonomous agents, which keep iterating until a goal is done.

For beginners, tool-using agents are the best starting point because they are easier to control and debug.

### 4. Recommended stack

For Python-based agents, a strong starting stack is:

- Python 3.9+.
- An LLM API or local model.
- A simple CLI or Streamlit UI.
- Tool functions in plain Python.
- Optional memory with files, SQLite, or a vector store.
- Optional frameworks like LangChain, CrewAI, or SDK-specific agent libraries.

For learning, it is best to build one agent **from scratch first** before using large frameworks.

### 5. Agent architecture

A simple agent loop looks like this:

1. Receive user goal.
2. Ask the model what to do next.
3. If a tool is needed, call it.
4. Send the tool result back to the model.
5. Repeat until the task is complete.

This “think → act → observe → repeat” pattern is the foundation of agentic AI.

### 6. Build a simple agent

Below is a **small from-scratch agent skeleton** in Python. It is not tied to any one framework, so you can understand the mechanics clearly.

python

```
import os
```

```
import json
```

```
# Example tools
```

```
def get_time(_=None):
```

```
    from datetime import datetime
```

```
    return datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
def add_numbers(args):
```

```
    a = args.get("a", 0)
```

## BrainStack

```
b = args.get("b", 0)
```

```
return a + b
```

```
TOOLS = {
```

```
    "get_time": get_time,
```

```
    "add_numbers": add_numbers
```

```
}
```

```
SYSTEM_PROMPT = """
```

```
You are a helpful AI agent.
```

```
You can reason step by step and use tools when needed.
```

```
When you want to use a tool, respond in JSON:
```

```
{"action": "tool_name", "args": {...}}
```

```
When finished, respond in JSON:
```

```
{"action": "final", "answer": "..."}  
"""
```

```
def mock_llm(messages):
```

```
    # Replace this with real LLM API call.
```

```
    last = messages[-1]["content"].lower()
```

```
    if "time" in last:
```

```
        return {"action": "get_time", "args": {}}
```

```
    if "add" in last:
```

```
        return {"action": "add_numbers", "args": {"a": 5, "b": 7}}
```

```
    return {"action": "final", "answer": "I can answer directly."}
```

```
def run_agent(user_input, max_steps=5):
```

```
    messages = [
```

```
        {"role": "system", "content": SYSTEM_PROMPT},
```

```
        {"role": "user", "content": user_input}
    ]

    for _ in range(max_steps):
        decision = mock_llm(messages)

        if decision["action"] == "final":
            return decision["answer"]

        tool_name = decision["action"]
        args = decision.get("args", {})

        if tool_name in TOOLS:
            result = TOOLS[tool_name](args)
            messages.append({"role": "assistant", "content": json.dumps(decision)})
            messages.append({"role": "tool", "content": str(result)})
        else:
            return "Tool not found."

    return "Agent stopped after max steps."

print(run_agent("What time is it?"))
print(run_agent("Please add numbers"))
```

### 7. How real LLM-based agents work

In a real agent, `mock_llm()` would be replaced by an API call to a model such as GPT, Claude, Gemini, or another provider. The model would return either a final answer or a structured tool call, and your Python code would execute that tool and feed the result back.

Structured output is important because agents must be machine-readable, not just conversational.

### 8. Add tools

Tools are just Python functions that do useful work. Common agent tools include:

- Web search.
- Calculator.
- File reader.
- Database query.
- Calendar access.
- API call.
- Code execution.
- Document retrieval.

Here is a cleaner tool example:

python

```
def calculator(args):  
    op = args.get("op")  
    a = args.get("a", 0)  
    b = args.get("b", 0)  
  
    if op == "add":  
        return a + b  
    if op == "sub":  
        return a - b  
    if op == "mul":  
        return a * b  
    if op == "div":  
        return a / b if b != 0 else "Division by zero"  
    return "Unknown operation"
```

Tools should be narrow, reliable, and easy to test.

### 9. Add memory

Memory lets an agent remember user preferences, previous tasks, or facts from earlier turns. Basic memory can be a Python dictionary or a JSON file, while advanced memory can use vector databases or retrieval systems.

Simple memory example:

```
python
```

```
memory = {}
```

```
def remember(key, value):
```

```
    memory[key] = value
```

```
    return f"Saved {key}"
```

```
def recall(key):
```

```
    return memory.get(key, "No memory found")
```

Memory is useful, but it should be controlled carefully so the agent does not rely on stale or incorrect information.

### 10. Multi-agent systems

A multi-agent system uses specialized agents for different responsibilities, such as planning, coding, searching, or reviewing. This architecture is useful when tasks are too large for one prompt or one model pass.

Example roles:

- Planner agent.
- Research agent.
- Coding agent.
- Reviewer agent.

This approach can improve task decomposition, but it also adds complexity and coordination overhead.

### 11. Guardrails

Guardrails are important because agents can make mistakes or take unsafe actions. Good guardrails include:

- Allowed tool list.
- Max step count.
- Human approval for risky actions.
- Input validation.
- Output validation.
- Logging and monitoring.

For production systems, always verify model outputs before acting on them.

### 12. Example: task agent

Here is a slightly more realistic pattern for a task agent:

python

```
def search_files(args):  
    folder = args.get("folder", ".")  
  
    import os  
  
    return os.listdir(folder)  
  
  
def write_file(args):  
    filename = args.get("filename")  
    content = args.get("content", "")  
    with open(filename, "w") as f:  
        f.write(content)  
  
    return f"Wrote to {filename}"
```

The agent can then choose between searching files or writing them depending on the user goal.

### 13. Best project ideas

Good beginner agent projects include:

- Personal assistant agent.
- Study planner agent.
- File organizing agent.
- Resume critique agent.
- Research summarizer agent.
- Coding helper agent.
- Calendar reminder agent.

These are practical because they require tools, memory, and structured decision-making.

### 14. How to learn by building

The best path is:

1. Build a rule-based agent.
2. Add tool calling.
3. Add memory.
4. Add retrieval.
5. Add a UI.
6. Add multi-agent routing.
7. Add deployment.

This order helps you understand what each layer contributes.

### 15. Common mistakes

Avoid these mistakes:

- Making the prompt too vague.
- Allowing too many tools.
- Skipping structured outputs.
- Not validating tool arguments.
- Building multi-agent systems too early.
- Trusting model output without checks.

Most failed agents are not “smart” enough in prompt design rather than in model quality.



### 16. Production considerations

For deployment, think about:

- Latency.
- Cost per step.
- Logging.
- Tool reliability.
- Error handling.
- Security.
- User permissions.
- Fallback behavior.

Production agents should be predictable, observable, and recoverable.

### 17. Conclusion

AI agents are built by combining a model with tools, memory, and a loop that repeatedly plans, acts, and checks results. The easiest way to learn them is to build a small Python agent from scratch, then gradually add tool use, memory, and multi-agent workflows.

### 18. Quick roadmap

- Learn structured outputs.
- Build one tool-using agent.
- Add memory.
- Add retrieval.
- Add multi-agent orchestration.
- Deploy a small agent app.